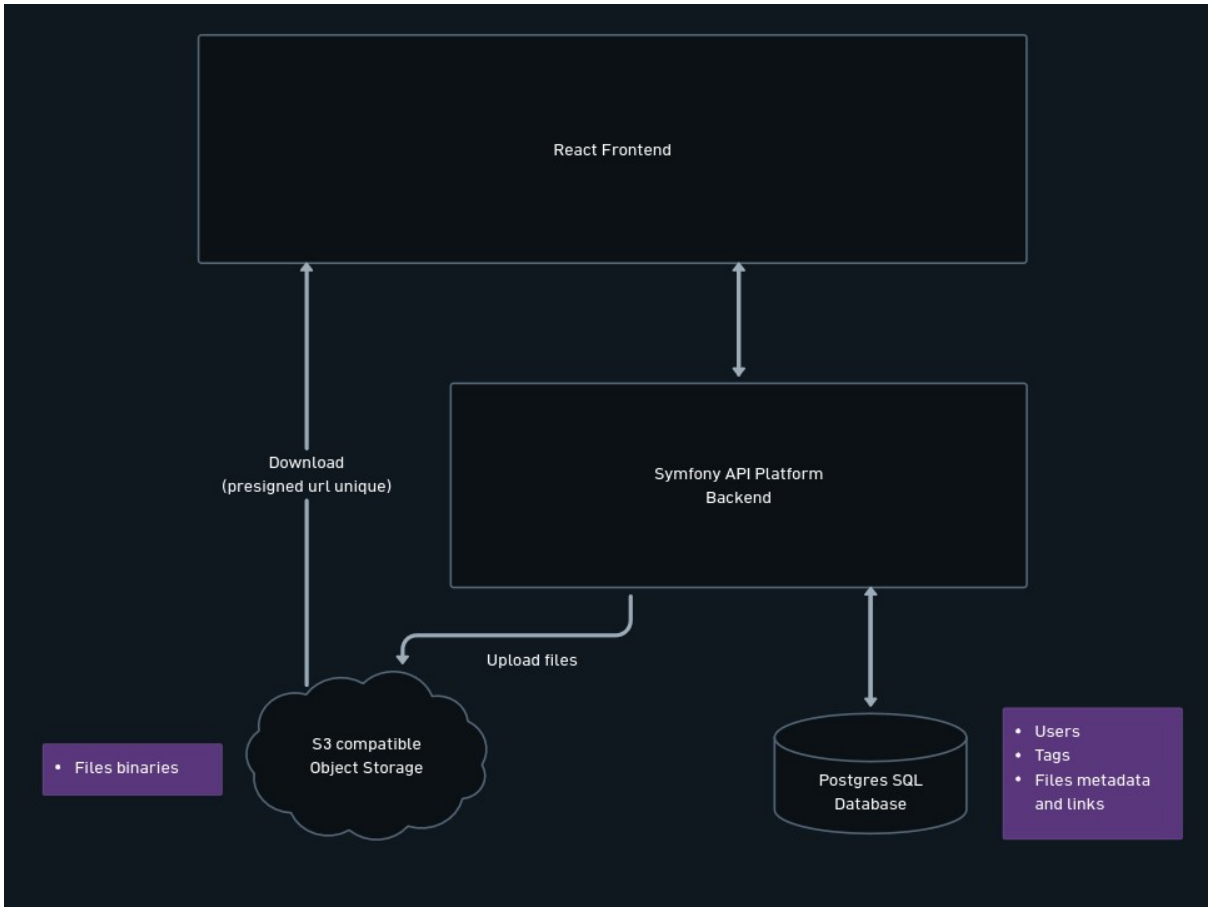
Documentation technique

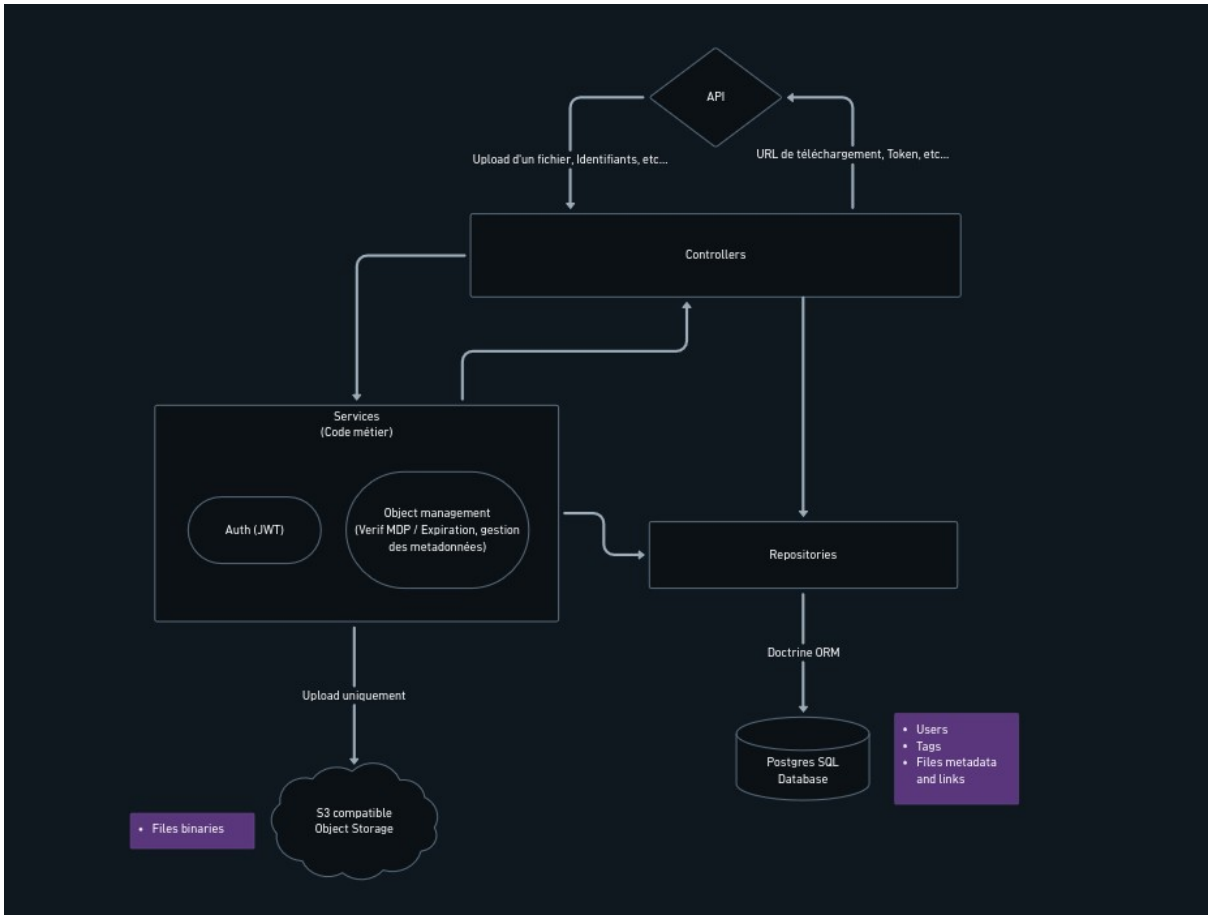
Architecture de l'application.....2
Choix technologiques justifiés.....5
Modèle de données.....10
Documentation d’API.....11
Où se situe la spécification.....11
Conventions communes.....11
Les routes.....12
Trois points qui expliquent la forme du contrat.....13
Codes de réponse.....13
Sécurité et gestion des accès.....14
Qualité, tests et maintenance.....15
Processus d’installation et d’exécution.....16
Utilisation de l’IA dans le développement.....17

Architecture de l'application

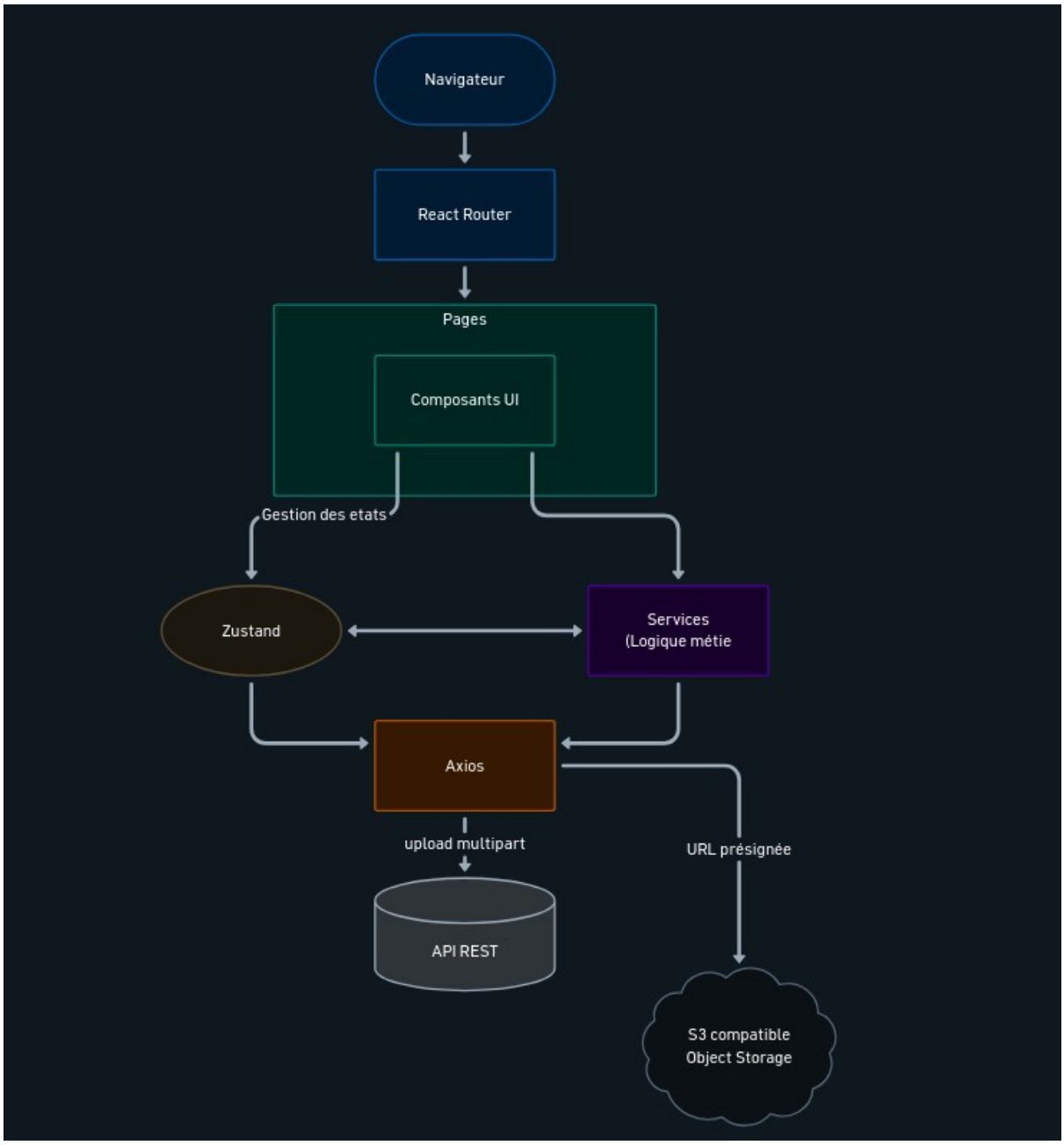
Architecture globale :



Backend :


API
Upload d'un fichier, Identifiants, etc...
URL de téléchargement, Token, etc...
Controllers
Services
(Code métier)
Auth (JWT)
Object management
(Verif MDP / Expiration, gestion
des metadonnées)
Repositories
Doctrine ORM
Upload uniquement
Postgres SQL
Database
Users
Tags
Files metadata
and links
S3 compatible
Object Storage
Files binaries

Frontend :


Navigateur
React Router
Pages
Composants UI
Gestion des etats
Zustand
Services
(Logique métie
Axios
upload multipart
URL présignée
API REST
S3 compatible
Object Storage

Choix technologiques justifiés

Briques structurantes

Élément	Technologie choisie	Alternatives	Justification
Back-end	PHP 8.4	Java, C#, TypeScript (Node)	C'est une techno que je maitrise déja. Le temps d'apprentissage d'un autre langage se prendrait sur les livrables évalués.
Framework back-end	Symfony 8.1 + API Platform 4.3	NestJS	Écosystème couvrant d'emblée JWT, validation, migrations et limitation de débit. OpenAPI généré depuis le code, donc un contrat d'API qui ne peut pas diverger de l'implémentation. NestJS était meilleur sur l'envoi de 1 Go et offrait un langage unifié, mais son coût d'apprentissage n'était pas finançable sur 6 semaines.
Langage front-end	TypeScript 6	JavaScript	Les types rendent le contrat d'API vérifiable à la compilation. `tsc -b` tourne à chaque build, en local et en CI.
Framework front-end	React 19 + Vite 8	Angular, Vue	Prototype non repris : la souplesse coûte moins cher ici que sur une base durable. Écosystème dense et démarrage immédiat. Angular apportait plus de garde-fous, écarté au profit de la vitesse de mise en route.
Base de données	PostgreSQL 16, relationnel	MongoDB	Le modèle est franchement relationnel : un utilisateur a N fichiers, un fichier a N tags, avec contraintes d'unicité et sélection par date d'expiration. MongoDB était défendable, sans bénéfice fonctionnel ici.
Stockage des fichiers	API S3, via Flysystem 3 ; implémentation SeaweedFS 4.47	AWS S3 réel, MinIO, Garage, LocalStack, adobe/s3mock	L'application s'adresse à Flysystem, jamais à une implémentation nommée. Passer en AWS S3 réel est un changement de variables d'environnement, sans une ligne de code applicatif. AWS S3 réel exigeait une carte bancaire et cassait l'installation automatisée.

Bibliothèques front-end

React n'impose rien : ces choix sont arrêtés en amont pour ne pas se décider au fil du développement.

Besoin	Choix	Justification
Routage	react-router 8	Standard de fait de l'écosystème React.
Appels HTTP	axios 1	Intercepteurs pour injecter le JWT, et suivi de progression de l'envoi, que `fetch` n'offre pas.
Formulaires et validation	react-hook-form 7 + zod 4	Les règles des spécifications deviennent déclaratives, donc testables une par une. Le serveur reste seul à faire foi, le front reproduit la règle pour afficher l'erreur avant l'envoi.
Styles	Tailwind CSS 4	Les maquettes existent en desktop et en mobile ; le responsive tient dans un préfixe de classe (`md:`).
État global client	zustand 5	Partage l'état d'authentification entre les pages sans prop drilling ni Context API verbeux. Son middleware `persist` gère la persistance du token.
État serveur	aucune bibliothèque	Un seul écran concerné. TanStack Query serait surdimensionné.
Icônes et police	lucide-react, @fontsource-variable/dm-sans	Alignement sur les maquettes, polices servies par l'application plutôt que par un CDN tiers.

Services tiers et infrastructure

Élément	Technologie choisie	Alternatives	Justification
Authentification	JWT signé RSA, LexikJWTAuthenticationBundle 3	OAuth2, sessions serveur	API sans état, donc horizontalement réplicable. Pas de fournisseur d'identité externe à faire vivre sur un prototype. Token d'une heure, non révocable : la durée courte est la contrepartie assumée.
Transit des fichiers	Envoi par l'API, téléchargement par URL présignée (60 s)	Tout par l'API ; tout par URL présignée	Configuration asymétrique et délibérée. L'envoi par l'API garde atomique le couple "objet écrit, métadonnées enregistrées". Le téléchargement par URL présignée évite à PHP de diffuser 1 Go, le contrôle d'accès restant côté serveur.
Conteneurisation	Docker Compose	Installation native	Docker est le seul prérequis : ni PHP, ni Node, ni PostgreSQL à installer. Back, front, base, stockage et outils de mesure démarrent par une commande.
Intégration continue	GitHub Actions, 3 workflows	GitLab CI, Jenkins	Le dépôt est sur GitHub. Les workflows rejouent lint, analyse statique, tests et couverture sur chaque pull request. La CI part d'un cache vide, ce qui en fait le détecteur de pannes d'infrastructure (voir plus bas).
Hébergement	non déployé, stack locale	PaaS, VPS	Hors périmètre du MVP. Les points à reprendre avant un déploiement réel sont listés dans SECURITY.md et MAINTENANCE.md.

Outils de développement

Élément	Technologie choisie	Alternatives	Justification
Versionnement	Git + GitHub, commits conventionnels, une branche et une pull request par ticket	GitLab	Chaque décision est rattachable à son ticket, et l'historique reste lisible.
Gestion de dépendances	Composer (PHP), npm (JS)	-	Fichiers de verrouillage versionnés : une installation reproduit exactement les versions d'une autre, et la relecture de leur diff fait la revue d'une montée de version.
Qualité du code back	PHP-CS-Fixer 3, PHPStan 2 au niveau max	-	Le style n'est jamais un sujet de relecture, et l'analyse statique au niveau le plus strict attrape les signatures qui changent lors d'une montée de version.
Qualité du code front	oxlint 1 (plugins react, typescript, oxc, jsx-a11y) + Prettier 3	ESLint	Configuration livrée par défaut par le template Vite, donc coût nul. Le plugin jsx-a11y couvre l'accessibilité, que React ne garantit pas. Écrit en Rust, il s'exécute en une fraction de seconde.
Tests	PHPUnit 13 + PCOV, Vitest 5 + Testing Library, Cypress 16	-	Trois niveaux, trois questions distinctes. Détail dans TESTING.md.
Mesures de performance	k6, Lighthouse 13.5, Monolog sur un canal metrics	Gatling, OctoPerf, Lighthouse CI	k6 est nommé par l'énoncé, tourne depuis une image Docker et son scénario est un fichier versionné, donc rejouable par un tiers. Gatling imposait une JVM et un DSL à apprendre ; OctoPerf, un compte externe et aucun scénario versionné. Lighthouse CI a été écarté pour ses 17 vulnérabilités transitives, dont 11 de sévérité haute, sans correctif non cassant.
Orchestration des commandes	un Makefile par projet	scripts npm/composer seuls	Mêmes verbes partout (make test, make lint, make audit), quel que soit le langage derrière.

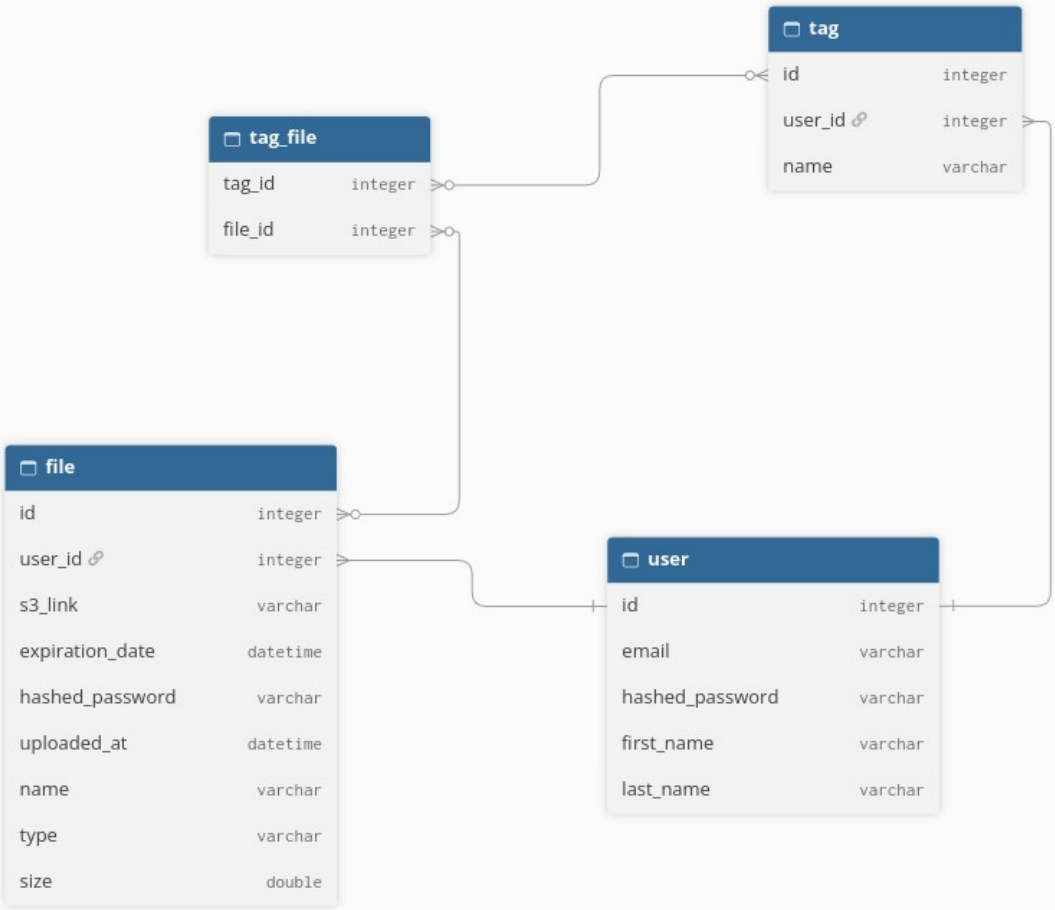
Une décision mise à l'épreuve

Le choix de stockage énonçait que l'application dépend de l'API S3 et non d'un fournisseur. Cette promesse a été vérifiée en cours de projet, et plus tôt que prévu.

MinIO était l'implémentation initiale. Son éditeur a progressivement abandonné l'édition communautaire, jusqu'au retrait des images des registres en septembre 2026. C'est la CI qui l'a détecté, sur un ticket qui ne touchait pas à l'infrastructure : dernier build vert le 23 septembre, premier échec le 28, sur un simple `unauthorized`. Les postes qui avaient l'image en cache ne voyaient rien.

Le remplacement par SeaweedFS n'a modifié **aucune ligne de code applicatif** : deux fichiers Compose et deux variables d'environnement. Le choix s'est fait sur essai et non sur documentation, chaque candidat devant créer un bucket, recevoir un objet, émettre une URL présignée puis servir cette URL en 200 avec le contenu attendu. SeaweedFS est le seul à avoir franchi les quatre étapes.

Modèle de données



Documentation d’API

Où se situe la spécification

La spécification **OpenAPI 3.1** est générée depuis le code par API Platform, elle ne peut donc pas diverger de l'implémentation. Stack démarrée, elle est servie à deux adresses :

Format	Adresse
Swagger UI, interactif	http://localhost:8080/api
Spécification JSON	http://localhost:8080/api/docs.jsonopenapi

Elle décrit 10 opérations et 12 schémas de données. Le test fonctionnel `OpenApiTest` épingle la description de chaque route : une opération ajoutée sans description correcte fait échouer l'intégration continue.

Le contrat détaillé en français, avec les règles métier que la spécification ne porte pas, est dans `docs/specs/API/contrat-api.md`. Le tableau ci-dessous en est la synthèse.

Conventions communes

- **Format** : JSON simple en entrée et en sortie, sauf l'envoi de fichier, en `multipart/form-data`. Pas d'enveloppe de pagination, une collection est un tableau.
- **Authentification** : JWT transmis dans l'en-tête `Authorization: Bearer <token>`, valable une heure.
- **Erreurs** : RFC 7807. Les champs fautifs sont listés sous `violations`, chacun avec son `propertyPath` et son message.

Les routes

Méthode et route	US	Auth	Requête	Réponse
Authentification				
POST /api/register	US03	non	{ email, plainPassword, firstName, lastName }	201 { id, email, firstName, lastName }
POST /api/login	US04	non	{ email, password }	200 { token }
Fichiers				
POST /api/files	US01, US07, US09	facultative	multipart : file, expiresInDays? (1 à 7), password?, tags?	201 { id, name, size, mimeType, downloadToken, expiresAt, hasPassword, tags }
GET /api/files	US05	requise	?tag= facultatif	200 tableau de fichiers du compte, avec sentAt et status
DELETE /api/files/{id}	US06	propriétaire	aucun	204
Tags				
POST /api/files/{id}/tags	US08	propriétaire	{ tag }, 30 caractères maximum	201 { id, tags }
PUT /api/files/{id}/tags/{tag}	US08	propriétaire	{ tag }, le nouveau nom	200 { id, tags }
DELETE /api/files/{id}/tags/{tag}	US08	propriétaire	aucun	200 { id, tags }
Téléchargement par lien public				
GET /api/downloads/{downloadToken}	US02	non	aucun	200 { name, size, mimeType, expiresAt, hasPassword } ; 410 si le lien est expiré ou inconnu
POST /api/downloads/{downloadToken}	US02, US09	non	{ password? }, requis si hasPassword	200 { presignedUrl, expiresIn }

Trois points qui expliquent la forme du contrat

L'authentification de `POST /api/files` est facultative, pas ignorée. Un envoi sans en-tête `Authorization` crée un fichier qui n'appartient à aucun compte (US07) : il n'apparaît dans aucun historique et les tags y sont refusés. Un en-tête présent mais invalide répond 401, il n'est jamais traité comme un envoi anonyme.

Le fichier n'est jamais servi par l'API. `POST /api/downloads/{downloadToken}` vérifie l'expiration puis le mot de passe, et renvoie une URL présignée S3 valable 60 secondes. Le contrôle d'accès reste côté serveur, sans que PHP ait à diffuser 1 Go.

`{tag}` est le nom du tag, pas un identifiant technique, parce que c'est ce que le front possède déjà dans les réponses des autres routes. Un nom est unique par compte : poser un nom déjà connu rattache le fichier au tag existant, ce qui est précisément ce qui fait fonctionner le filtre de l'historique.

Codes de réponse

Code	Quand
400	Requête illisible, corps JSON mal formé ou champ de connexion absent.
401	Authentification manquante ou invalide, ou mot de passe de téléchargement incorrect. Un mot de passe faux et une adresse inconnue renvoient le même message, afin de ne pas révéler quels comptes existent.
403	Action sur une ressource dont l'utilisateur n'est pas propriétaire.
404 / 410	Ressource introuvable, tag absent du fichier, ou lien de téléchargement expiré. Le 410 ne distingue pas un lien expiré d'un lien qui n'a jamais existé.
422	Validation : email déjà utilisé, mot de passe trop court ou trop faible, taille supérieure à 1 Go, extension interdite, expiration au-delà de 7 jours, tags soumis sans compte, tag vide ou trop long.
429	Plus de cinq tentatives échouées en quinze minutes, à la connexion ou sur le mot de passe d'un lien. L'en-tête `Retry-After` donne le délai en secondes.

Sécurité et gestion des accès

Authentification. Elle repose sur un jeton JWT signé par une clé privée. Le serveur ne garde aucune session : il lui suffit de vérifier la signature du jeton que le navigateur présente à chaque requête. Le jeton vaut une heure, et ne peut pas être annulé avant son échéance ; cette durée courte est la contrepartie assumée.

Permissions. L'application ne distingue pas de rôles, il n'y a ni administrateur ni modérateur. Les droits se décident route par route, certaines étant publiques par conception.

Action	Qui y a droit
Déposer un fichier	Tout le monde, avec ou sans compte
Consulter son historique	Un compte connecté, pour ses fichiers seulement
Supprimer un fichier, gérer ses étiquettes	Son propriétaire, et lui seul
Télécharger un fichier	Quiconque détient le lien, et son mot de passe s'il en a un

Mots de passe. Aucun n'est conservé en clair, seule une empreinte irréversible l'est. Celui d'un compte exige seize caractères et une robustesse minimale, soit plus que les huit demandés par les spécifications : l'inscription est la porte d'entrée de tout le reste. Celui que l'on peut apposer sur un fichier partagé se contente de six caractères, puisqu'il ne protège qu'un fichier pour quelques jours.

Validation des entrées. Les règles sont écrites côté serveur, seul à faire foi ; le formulaire les reproduit pour afficher l'erreur avant l'envoi. Les requêtes passent par un ORM qui sépare l'instruction de ses paramètres, ce qui ferme la porte aux injections, et le front neutralise par défaut tout contenu inséré dans la page.

Limites sur les dépôts. La taille est plafonnée à 1 Go, et ce plafond est aligné dans l'application, dans PHP et dans le serveur web : un fichier trop gros est refusé par un message clair, pas par une erreur technique. Douze extensions exécutables sont refusées, et un fichier expire au bout de sept jours au plus.

Liens de partage. Un lien contient 32 caractères tirés au hasard, mais il n'est pas l'adresse de téléchargement : le serveur vérifie d'abord l'expiration et le mot de passe, puis fabrique une adresse temporaire valable soixante secondes vers l'espace de stockage. Le contrôle d'accès reste donc côté serveur. Cinq tentatives par quart d'heure sont tolérées sur un mot de passe, de lien comme de compte.

Ce qui relève du déploiement. Le prototype tourne en local : les échanges ne sont pas chiffrés, et HTTPS reste à mettre en place, comme le plafonnement des dépôts sans compte et le remplacement des identifiants de développement par de vrais secrets. La liste complète est dans SECURITY.md.

Qualité, tests et maintenance

Quatre documents du dépôt portent le suivi de qualité.

TESTING.md décrit le plan de tests : une ligne par fonctionnalité, avec son critère d'acceptation et le fichier qui le vérifie. Trois niveaux répondent à trois questions : les tests unitaires isolent une classe et ses règles de décision, les tests fonctionnels envoient de vraies requêtes pour éprouver le routage, la sécurité et la validation, et les tests de bout en bout pilotent un navigateur sur l'application entière.

SECURITY.md rend compte des audits. Deux commandes interrogent les bases de vulnérabilités publiques, pour les bibliothèques du projet et pour les images de conteneurs. L'audit d'octobre 2026 a relevé deux failles dans le socle applicatif, dont une exploitable par n'importe quel visiteur ; les deux ont été corrigées par une montée de version. Côté conteneurs, aucune faille ne concerne le code du projet, toutes venant des systèmes de base des images officielles. Le relevé sert désormais de point de comparaison d'un scan au suivant.

PERF.md rassemble trois mesures. Le dépôt de fichier est la seule opération où la taille pèse sur le code, le téléchargement partant en direct du stockage : c'est lui qui a été mis en charge. Un fichier d'un gigaoctet est accepté et confirmé en 63 secondes, sans qu'aucune limite du serveur soit atteinte. Le serveur journalise enfin chaque requête avec sa durée et son volume. La campagne a aussi révélé un vrai défaut, corrigé depuis : le formulaire et le serveur ne comptaient pas le gigaoctet de la même manière, si bien qu'un fichier accepté à l'écran pouvait être refusé une minute plus tard.

MAINTENANCE.md fixe les procédures de mise à jour. Les versions exactes de chaque dépendance sont enregistrées dans le dépôt, si bien qu'une installation reproduit à l'identique n'importe quelle autre. Le rythme est défini : un audit chaque semaine, les correctifs chaque mois, les versions majeures chaque trimestre et une à la fois, et sans délai pour toute faille atteignable depuis l'extérieur. Les mises à jour se font par famille de bibliothèques plutôt qu'en bloc, pour qu'un incident reste attribuable. Le document consigne enfin un épisode marquant : le service de stockage retenu au départ a vu ses images disparaître des serveurs publics en cours de projet, panne détectée par l'intégration continue et réparée sans qu'une ligne de code change.

Processus d’installation et d’exécution

Prérequis. Docker et le plugin Docker Compose, et rien d'autre. Ni PHP, ni Node, ni PostgreSQL ne sont à installer : chaque brique tourne dans son conteneur, et les versions dont dépend le projet sont fixées par les images. Seule réserve, les tests de bout en bout ne s'exécutent que sous Linux.

Première installation. Trois commandes, depuis la racine du dépôt.

```
cd DataShare_API
make init
cd ..
docker compose up -d
```

make init installe les dépendances du serveur, produit les clés qui signeront les jetons de connexion et met la base de données à jour ; la commande suivante démarre l'ensemble. Compter une minute avant que l'interface réponde, le temps qu'elle installe ses propres dépendances. Ensuite, **docker compose up -d** et **docker compose down** suffisent au quotidien.

L'application est alors accessible à ces adresses.

Quoi	Adresse
Application	http://localhost:5173
API, documentation interactive comprise	http://localhost:8080/api
Base de données	http://localhost:8081
Espace de stockage	http://localhost:8333

Commandes courantes. Chaque projet du dépôt possède son propre fichier de commandes, dont la liste est commentée sur place. Les mêmes verbes servent partout, quel que soit le langage derrière : **make test**, **make coverage**, **make lint**, **make migrate** et **make audit**.

Variables d'environnement. Les valeurs par défaut sont livrées avec le projet et suffisent en local, de sorte qu'un poste neuf démarre sans réglage. Ce sont des valeurs de développement, à remplacer par de vrais secrets pour tout déploiement réel. Ce qui est propre à un poste va dans un fichier **.env.local** placé à côté, jamais versionné.

Variable	À quoi elle sert
JWT_PASSPHRASE	Protège les clés qui signent les jetons de connexion. Créée automatiquement à l'installation.
VITE_API_URL	Adresse de l'API appelée par le navigateur.

Utilisation de l’IA dans le développement

La posture adoptée. Elle est triple, et c'est ce qui la distingue du simple binômage. L'IA a d'abord été un partenaire de discussion, chargé de contredire mes idées plutôt que de les exécuter. Elle a ensuite tenu le rôle d'un développeur junior à qui l'on assigne des tickets. Elle a enfin servi de relecteur, pour chercher les failles, les mauvaises pratiques et les manques dans la documentation. Ces trois rôles sont restés distincts tout au long du projet, et je suis resté l'unique décideur.

Les tâches confiées. La collaboration a suivi l'ordre du projet. Au départ, l'IA a servi d'interlocuteur pour arrêter le choix du socle technique et pour mettre à l'épreuve mes décisions d'architecture, chaque option devant résister à la contradiction avant d'être retenue. Elle a ensuite rédigé les tickets, à partir d'une liste de travaux que je lui ai fournie et des décisions d'architecture déjà prises ; ces tickets ont fait l'objet d'allers-retours de ma part, pour en corriger la pertinence et en redécouper plusieurs. Elle a enfin pris en charge les tickets un par un.

La supervision et les corrections. Chaque ticket s'est déroulé en deux temps. L'IA produisait d'abord un plan d'implémentation, que je relisais et corrigeais avant toute écriture de code. L'implémentation ne commençait qu'une fois ce plan validé, et se terminait par un test manuel de ma part. L'IA créait ensuite la branche et la demande de fusion, mais **n'a jamais été autorisée à fusionner quoi que ce soit**. Chaque fusion a été faite par un humain (moi), après relecture des modifications ligne à ligne, et à la double condition que le test manuel et l'intégration continue soient l'un et l'autre au vert. Cette règle n'a pas souffert d'exception.

Les apports. Le gain de temps a été considérable, au point que je ne pense pas que le projet aurait tenu dans les délais sans cette aide. La sécurité en est sortie renforcée, un relecteur systématique repérant ce qu'une relecture humaine laisse passer. L'IA m'a par ailleurs fait découvrir des technologies et des services que je ne connaissais pas et qui ont simplifié certaines implémentations. Elle a enfin rendu possible une couverture de tests systématique pour un effort réduit, là où elle aurait sinon été la première chose sacrifiée.

Les limites. Deux, et elles tiennent au même phénomène. L'apprentissage des technologies est moins profond qu'il ne l'aurait été en écrivant soi-même : une part de confiance accordée au code produit remplace la compréhension qu'on acquiert en se trompant. Et la quantité de code produite en peu de temps crée

une surcharge cognitive qui limite la capacité à garder un œil critique sur l'intégralité de ce qui est écrit. C'est précisément ce que la relecture obligatoire avant fusion, le test manuel et l'intégration continue servent à contenir, sans jamais le supprimer complètement.